

Ibad ullah
25K-6544

a) A static array uses **compile-time allocation**, meaning its size is fixed before the program runs.

The number of students is unknown at the start

It may waste memory if fewer students enroll or it may overflow if more students enroll than expected

It lacks flexibility since the size cannot be changed during execution

runtime allocation is more suitable because it allows memory to be allocated based on actual input.

b)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of students: ");
```

```
    scanf("%d", &n);
```

```
    float *grades = (float *)malloc(n * sizeof(float));
```

```
    if (grades == NULL) {
```

```
        printf("Memory allocation failed!\n");
```

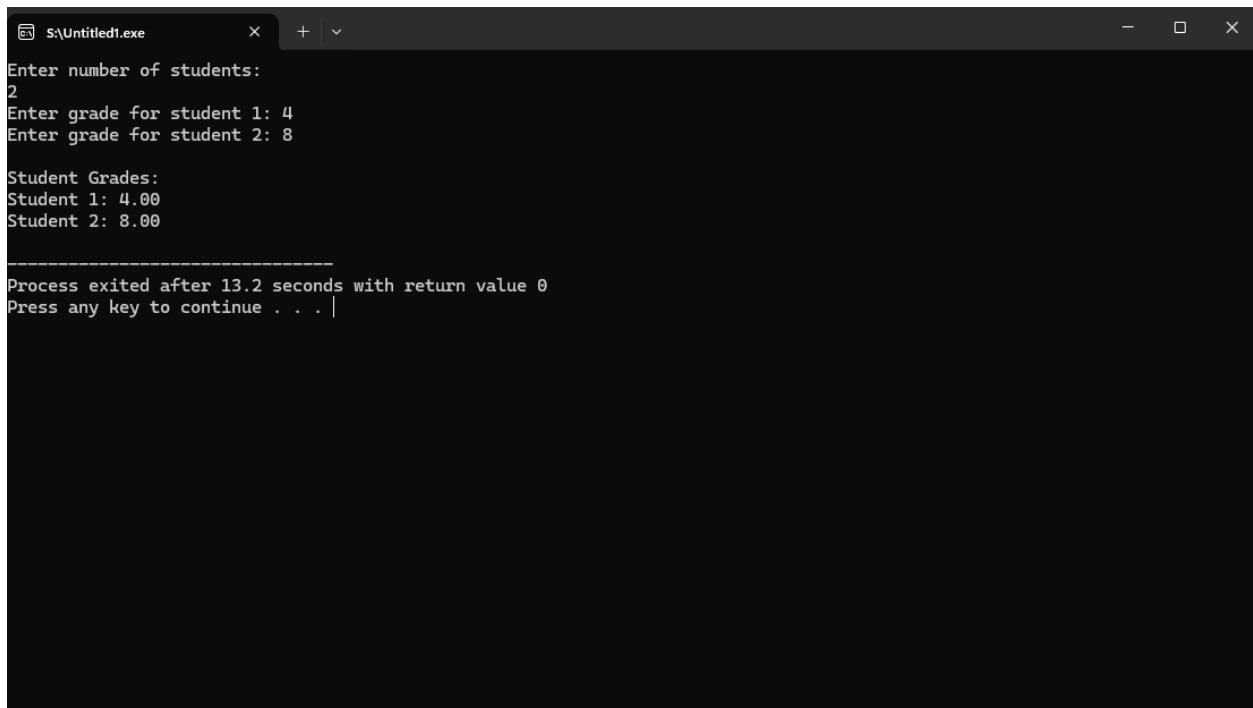
```
        return 1;
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

Ibad ullah
25K-6544

```
        printf("Enter grade for student %d: ", i + 1);  
        scanf("%f", &grades[i]);  
    }  
  
    printf("\nStudent Grades:\n");  
    for (int i = 0; i < n; i++) {  
        printf("Student %d: %.2f\n", i + 1, grades[i]);  
    }  
  
    free(grades);  
  
    return 0;
```



```
S:\Untitled1.exe  
Enter number of students:  
2  
Enter grade for student 1: 4  
Enter grade for student 2: 8  
  
Student Grades:  
Student 1: 4.00  
Student 2: 8.00  
  
-----  
Process exited after 13.2 seconds with return value 0  
Press any key to continue . . . |
```

c)

If `free(grades)` is not called, the allocated memory remains reserved even after it is no longer needed. This leads to a memory leak.

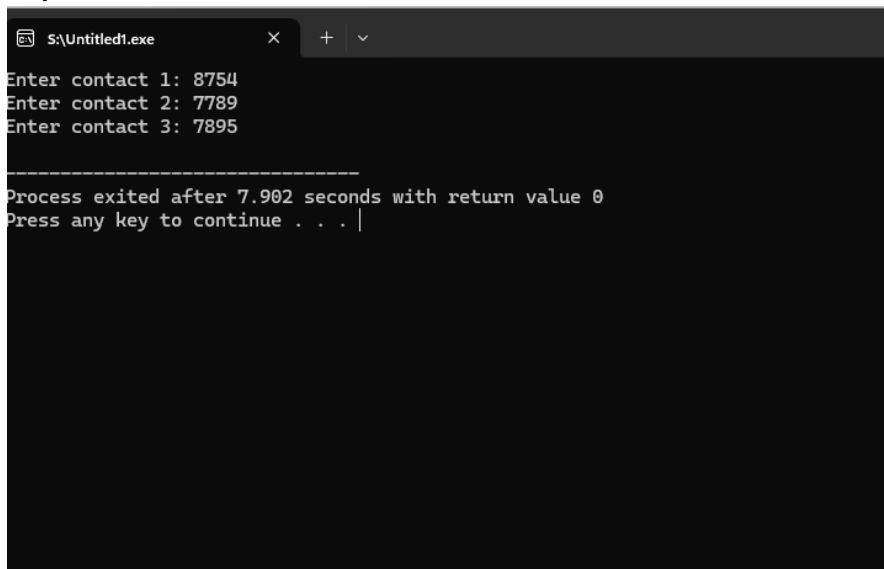
Ibad ullah
25K-6544

Q2)

a) Code:

```
#include <stdlib.h>
```

```
int main() {  
    int *contacts = (int *)malloc(3 * sizeof(int));  
  
    if (contacts == NULL) {  
        printf("Memory allocation failed!\n");  
        return 1;  
    }  
  
    for (int i = 0; i < 3; i++) {  
        printf("Enter contact %d: ", i + 1);  
        scanf("%d", &contacts[i]);  
    }  
}
```



```
S:\Untitled1.exe  
Enter contact 1: 8754  
Enter contact 2: 7789  
Enter contact 3: 7895  
-----  
Process exited after 7.902 seconds with return value 0  
Press any key to continue . . . |
```

b)

code:

```
#include <stdlib.h>
```

```
int main() {  
    int *contacts = (int *)malloc(3 * sizeof(int));
```

Ibad ullah
25K-6544

```
if (contacts == NULL) {  
    printf("Memory allocation failed!\n");  
    return 1;  
}
```

```
for (int i = 0; i < 3; i++) {  
    printf("Enter contact %d: ", i + 1);  
    scanf("%d", &contacts[i]);  
}
```

```
int *temp = (int *)realloc(contacts, 5 * sizeof(int));
```

```
if (temp == NULL) {  
    printf("Reallocation failed!\n");  
    free(contacts);  
    return 1;  
}
```

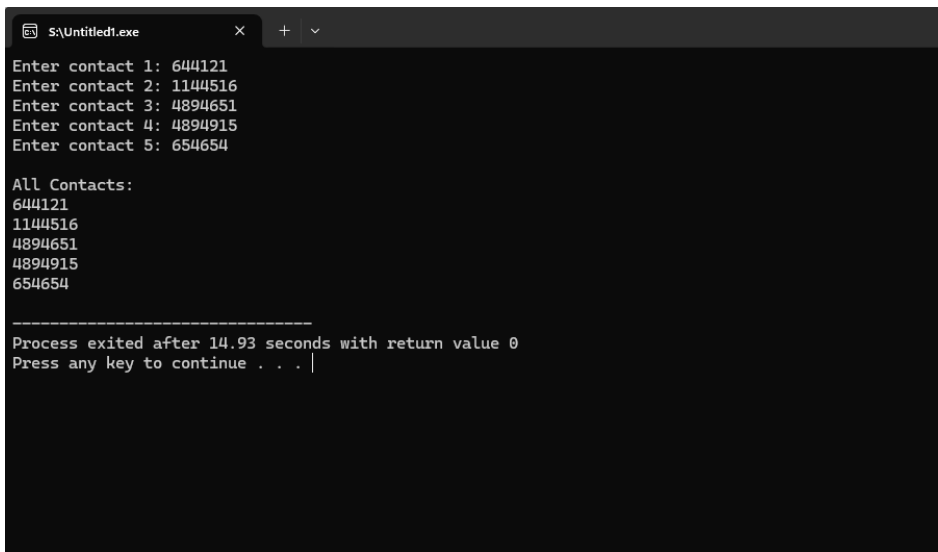
```
contacts = temp;
```

```
for (int i = 3; i < 5; i++) {  
    printf("Enter contact %d: ", i + 1);  
    scanf("%d", &contacts[i]);  
}
```

Ibad ullah
25K-6544

```
printf("\nAll Contacts:\n");  
for (int i = 0; i < 5; i++) {  
    printf("%d\n", contacts[i]);  
}
```

```
free(contacts);  
return 0;
```



```
S:\Untitled1.exe  
Enter contact 1: 644121  
Enter contact 2: 1144516  
Enter contact 3: 4894651  
Enter contact 4: 4894915  
Enter contact 5: 654654  
  
All Contacts:  
644121  
1144516  
4894651  
4894915  
654654  
  
-----  
Process exited after 14.93 seconds with return value 0  
Press any key to continue . . . |
```

c)

to avoid realloc to go null,

```
int *temp = realloc(contacts, new_size);
```

```
if (temp == NULL) {  
    printf("Reallocation failed!\n");  
    free(contacts);  
    return 1;
```

Ibad ullah
25K-6544

```
}
```

```
contacts = temp;
```

what is dangling pointer :

A dangling pointer is a pointer that still points to a memory location that has already been freed.

How to handle it :

After using free set the pointer to null.

Q3)

a)

malloc() allocates memory but leaves it uninitialized (garbage values)

Calloc() allocates memory and initializes all values to 0

b)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of sensor readings: ");
```

```
    scanf("%d", &n);
```

```
    float *buffer = (float *)calloc(n, sizeof(float));
```

```
    if (buffer == NULL) {
```

Ibad ullah
25K-6544

```
    printf("Memory allocation failed!\n");  
    return 1;  
}
```

```
printf("\nInitial values:\n");  
for (int i = 0; i < n; i++) {  
    printf("%.2f ", buffer[i]);  
}
```

```
printf("\n\nEnter sensor readings:\n");  
for (int i = 0; i < n; i++) {  
    scanf("%f", &buffer[i]);  
}
```

```
printf("\nSensor Data:\n");  
for (int i = 0; i < n; i++) {  
    printf("%.2f ", buffer[i]);  
}
```

```
free(buffer);  
return 0;
```

Ibad ullah
25K-6544

```
S:\Untitled1.exe X + v
Enter number of sensor readings: 5

Initial values:
0.00 0.00 0.00 0.00 0.00

Enter sensor readings:
1.2
5.4
4.3
1.0
4.2

Sensor Data:
1.20 5.40 4.30 1.00 4.20
-----
Process exited after 17.2 seconds with return value 0
Press any key to continue . . . |
```

c)

malloc()

Values using malloc

-34567.23 0.00 9.87 12345.67 -0.12

Calloc()

Values using calloc

0.00 0.00 0.00 0.00 0.00